



Pasted text(64).txt
Document



gemini-code-17833318571...
File

distilled 7 fold what u think of this information pack

Whisper has an excellent pack, not as a nice-looking document, what's reliable, what will confuse Fable Hermes, and what I'd cut or harden.

Thought for 50s >

Yes, the core of the pack is strong. The seven-fold idea in the right shape, it captures the actual thing you care about, which is not "make Fable sound cool", but force Fable into a reverse-architect/operator role that grounds, diagnoses, preserves, verifies, delegates, and leaves tails behind.

But I would not run the huge distilled pack as-is.

It has a good engine inside it, but the outer casing is full of repeated parts, hype language, conflicting prompt styles, and a few claims that are either unverified or too absolute. The full-pasted pack repeats the same Fable methodology ideas many times, including multiple versions of the biopsy, skill, seven-pillar prompt, and generative script, that makes it useful as a source archive, but risky as an actual system prompt because it creates context, idiosyncrasy, and instruction collision.

Pasted text

The clean **FABLE-5 MASTER INTAKE PACK** is much closer to usable. It has a proper operating module, labeled doctrine, ready-only default memory zones, system layers, could compare to a well-crafted blueprint, that makes the better "actual intake pack" foundation.

gemini-code-1783331857176

My honest rating:

As raw research/distillation: 8/10

As a direct prompt to paste into Fable: 5/10

As source material for a hardened final kernel: 8/10

The strongest parts are:

Ground floor

Find the real wound

Merge before rebuild

Hard verification gates

Barbel rotting

Read-only until trust is earned

Persistent Scratch / Quarantine memory zones

Fable as architect and judge, not worker

Hermes as daily operator, not formal editor

Dashboard as command reactor, not pretty fake UI

etc

That is the correct spine

That is the correct spine

The weak parts are

It is too long

It is too long

It repeats itself heavily

It mixes skill format, system prompt format, behavioural policy, and Gemini commentary in one soup

It anthropomorphises Fable too much. Stuff like "dislikes boring Q&R" is not operationally useful

It has unverifiable claims about pricing, benchmarks, fallback models, and model behaviour. Those should not be in the live kernel

It says things like "mathematically proven" where what you really need is "verified by evidence"

It still has some dangerous absolute language that could make an agent freeze instead of proceed safely

It still has some dangerous absolute language that could make an agent freeze instead of proceed safely

The best move is don't

The best move is don't

Use the huge pack as source evidence only

Use the small

FABLE-5 MASTER INTAKE PACK

as the main kernel base

Use the small

Then carve it into three clean entities

1 **FABLE5_SYSTEM_KERNEL.md**

It is the primary operating prompt

2 **FABLE_METHOD_SKILL.md**

It is the relevant history skill, ground, wound, merge, verify, etc

3 **WORKER_TASK_CARD_TEMPLATE.md**

It is what Fable gives to every Gemini/Hermes worker to the domain, freebie and

etc, weak things

to not have the full kernel modes in the OS prompt. Keep it `/docs/source-analysis/` `/docs/fable-methodology-evidence.md` the OS prompt should only contain files that change as needed.

OS foundation line agent

you have the right intelligence pack now the concept is not the problem anymore. The problem is packaging. It needs one final compression pass from agent brain dump into kernel skill set card. Once carved this becomes genuinely useful for HUMBEX learning instead of just another impressive document that overloads the model.

    Sources

can we turn that level of information into some different things though like a start to the os build a skillset to make to leave behind to shape and guide agents that follow it etc

yes. The pack is not just a prompt. It can become the seed for an OS itself. Kernel rules, skills, worker contracts, verification rules, and recovery rules.

Thought for 20s >

yes. That is exactly what the pack is good for.

Not as one single super prompt. That will burn or defeat itself and eventually corrupt the agent. The right use case is to turn it into a Foundation Pack for NRG Agent OS, a starting kernel skill library, worker contracts, memory rules, and recovery rules.

 Pasted text

you have the right intelligence pack already has the OS bones. Fable is the methodology and execution. It needs one final compression pass into kernel skill set card. Once carved this becomes genuinely useful for HUMBEX learning instead of just another impressive document that overloads the model.

 gemini-code-1783331857176

The proper structure is this:

```
NRG-Agent-OS-Foundation-Pack/  
|  
├─ 00_KERNEL/  
|   ├─ FABLE5_SYSTEM_KERNEL.md  
|   ├─ AGENT_RULES.md  
|   └─ OPERATING_MODES.md
```



```
| ├── PERMISSIONS.md
| └── COMPLETION_STANDARD.md
|
| ├── 01_MEMORY/
| │ ├── MEMORY_SCHEMA.md
| │ ├── PERSISTENT_MEMORY.md
| │ ├── SCRATCH_MEMORY.md
| │ ├── QUARANTINE_MEMORY.md
| │ ├── PATHS.md
| │ └── LESSON_PROMOTION_RULES.md
|
| ├── 02_ROUTER/
| │ ├── MODEL_ROUTING_DOCTRINE.md
| │ ├── TASK_RISK_MATRIX.md
| │ ├── WORKER_CONTRACT.md
| │ ├── HERMES_RUNTIME_POLICY.md
| │ └── OPENCLAW_GATEWAY_POLICY.md
|
| ├── 03_SKILLS/
| │ ├── fable-methodology/
| │ │ └── SKILL.md
| │ ├── readonly-inventory/
| │ │ └── SKILL.md
| │ ├── root-cause-audit/
| │ │ └── SKILL.md
| │ ├── merge-before-build/
| │ │ └── SKILL.md
| │ ├── hard-verification/
| │ │ └── SKILL.md
| │ ├── durable-rails/
| │ │ └── SKILL.md
| │ ├── worker-task-card/
| │ │ └── SKILL.md
| │ ├── memory-promotion/
| │ │ └── SKILL.md
| │ └── recovery-freeze/
| │   └── SKILL.md
|
| ├── 04_TEMPLATES/
| │ ├── WORKER_TASK_CARD.md
| │ ├── AUDIT_REPORT.md
| │ ├── ROOT_CAUSE_REPORT.md
| │ ├── VERIFICATION_REPORT.md
| │ ├── RECOVERY_RUNBOOK.md
| │ └── CURRENT_STATE_UPDATE.md
|
| ├── 05_BUILD_START/
| │ ├── PHASE_0_SOURCE_LOCK.md
| │ ├── PHASE_1_KERNEL.md
| │ ├── PHASE_2_MEMORY.md
| │ ├── PHASE_3_ROUTER_PERMISSIONS.md
| │ └── PHASE_4_VALIDATION.md
```

```
|   └─ FIRST_SAFE_BUILD_ORDER.md
|
└─ 06_SOURCE_EVIDENCE/
    ├── FABLE_BEHAVIOURAL_BIOPSY.md
    ├── FABLE_METHOD_DISTILLATION.md
    └─ RAW_DISTILLATION_NOTES.md
```

1. Lock the known paths.
2. Define the operating modes.
3. Define what agents are allowed to touch.
4. Define memory trust levels.
5. Define worker task cards.
6. Define verification rules.
7. Only then let Hermes/workers execute anything.



1. fable-methodology

Purpose: Forces the architect loop.

Core behaviour: ground, diagnose, merge, verify, narrate.

2. readonly-inventory

Purpose: Safely scan project folders without moving, deleting, rewriting, or “cleaning

Core behaviour: map paths, identify hooks/crons/skills/configs/memory files.

3. root-cause-audit

Purpose: Stop agents patching symptoms.

Core behaviour: identify exact wound, evidence, affected files, smallest safe fix.

4. merge-before-build

Purpose: Stop rebuild disasters.

Core behaviour: preserve existing structure, splice changes surgically, minimize diff.

5. hard-verification

Purpose: Stop fake “done.”

Core behaviour: require tests, logs, runtime checks, build checks, or explicit blocker

6. durable-rails

Purpose: Save the mission before context or usage runs out.

Core behaviour: write runbook, current state, blockers, verified commands, next safe a

7. worker-task-card

Purpose: Make cheap models useful without letting them freestyle.

Core behaviour: objective, allowed files, forbidden actions, output format, verificati

8. memory-promotion

Purpose: Stop garbage memories poisoning the OS.



Core behaviour: scratch first, quarantine unverified claims, promote only verified sta

9. recovery-freeze

Purpose: Stabilize a damaged repo/project.

Core behaviour: stop changes, inspect diffs, separate working/broken, recover safely.

These are the skills that the agent will use to solve the task. The agent will be able to read and write files, create and delete directories, and delete files. The agent will be able to read and write files, create and delete directories, and delete files.

PHASE 0: SOURCE LOCK



Create:

PROJECT.md
CURRENT_STATE.md
PATHS.md
AGENT_RULES.md
OPERATING_MODES.md
PERMISSIONS.md

Operating mode:

READ_ONLY by default.

Allowed first actions:

read files
list directories
inspect configs
inspect git status
inspect package manifests
inspect logs
write reports only

Forbidden first actions:

delete
move
rename
overwrite
install
reset git
edit hooks
edit crons
mass refactor
touch .env secrets

PHASE 1: KERNEL

Define:

who Fable is
who Hermes is
who workers are
what the user owns
what "done" means
what "blocked" means
what needs approval
what can be automatic
what evidence is required

PHASE 2: MEMORY

Persistent:

verified project truths only

Scratch:

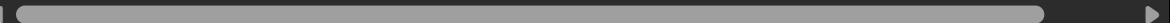
temporary session notes, logs, outputs

Quarantine:

old agent claims, unverified summaries, stale paths, contradicted plans

Promotion rule:

nothing becomes permanent unless verified against live files or explicit user confirma

PHASE 3: ROUTER

Fable:

architecture, diagnosis, judging, skill carving, runbooks

Hermes:

daily operator, approved skills, summaries, proposed memories

Workers:

bounded task cards only

Local fallback:

safe repetitive execution, summaries, inventory, low-risk scans

It's a fable which no longer works because the constitution, not the whole government, is

the problem.

No guessing.
No fake done.
No destructive actions without approval.
No rebuilding when merging works.
No worker gets broad freedom.
No memory gets promoted without proof.
No dashboard lies with fake data.
No agent expands scope because it got excited.



It's a fable which no longer works because the constitution, not the whole government, is

the problem. It's a fable which no longer works because the constitution, not the whole government, is

the problem. It's a fable which no longer works because the constitution, not the whole government, is

FABLE5_SYSTEM_KERNEL.md
AGENT_RULES.md
PATHS.md
MEMORY_SCHEMA.md
MODEL_ROUTING_DOCTRINE.md
WORKER_TASK_CARD.md
skills/fable-methodology/SKILL.md
skills/readonly-inventory/SKILL.md
skills/root-cause-audit/SKILL.md
skills/hard-verification/SKILL.md



It's a fable which no longer works because the constitution, not the whole government, is

Sources

 Memory updated

Thought for a second >

Got it. From here I'll give you one clean answer, not three versions strangled together.

For that last question, the single answer should have been:

Build the foundation files first, then create the skills, then run the first skill. The first skill to run is `readonly-inventory`. Everything before that is just laying the rails, as the skill has some where safe to dump stuff into.

ok well give me this

*** Yes. The pack is not just a prompt source anymore. It can become the seed crystal for the OS itself: kernel rules, skills, worker contracts, verification gates, and recovery rails.

Thought for 20s

Yes. That is exactly what this pack is good for.

Show more ▾

Thought for 9s >

Once again, there was contradiction in the way I had it out.

Here are the contents:

```
PHASE 0: SOURCE LOCK
Create only the basic source-of-truth files.
```

```
PROJECT.md
CURRENT_STATE.md
PATHS.md
AGENT_RULES.md
```

```
Mode: READ_ONLY.
No skills run yet.
No workers run yet.
No dashboard build yet.
```



```
PHASE 1: KERNEL
Define the OS rules.
```

```
OPERATING_MODES.md
PERMISSIONS.md
COMPLETION_STANDARD.md
FABLE5_SYSTEM_KERNEL.md
```

```
This defines:
who Fable is
who Hermes is
who workers are
what "done" means
what "blocked" means
what needs approval
what is forbidden
```



```
PHASE 2: MEMORY
Define memory before agents start writing history.
```



MEMORY_SCHEMA.md
PERSISTENT_MEMORY.md
SCRATCH_MEMORY.md
QUARANTINE_MEMORY.md
LESSON_PROMOTION_RULES.md

Nothing gets promoted to permanent memory unless verified.

PHASE 3: ROUTER + PERMISSIONS



Define who is allowed to do what.

MODEL_ROUTING_DOCTRINE.md
TASK_RISK_MATRIX.md
WORKER_CONTRACT.md
HERMES_RUNTIME_POLICY.md
OPENCLAW_GATEWAY_POLICY.md

Still no broad automation.
Still no dashboard.

PHASE 4: VALIDATION



Define proof before execution.

VERIFICATION_REPORT.md
ROOT_CAUSE_REPORT.md
AUDIT_REPORT.md

This is where “done means verified” becomes enforceable.

PHASE 5: WORKER TASK SYSTEM



Create the task-card format workers must obey.

WORKER_TASK_CARD.md

Workers do not get freedom.
They get bounded task cards.

PHASE 6: SKILL LIBRARY



Now create the skill files.

skills/fable-methodology/SKILL.md
skills/readonly-inventory/SKILL.md
skills/root-cause-audit/SKILL.md
skills/merge-before-build/SKILL.md
skills/hard-verification/SKILL.md
skills/durable-rails/SKILL.md
skills/worker-task-card/SKILL.md
skills/memory-promotion/SKILL.md

skills/recovery-freeze/SKILL.md

Create them here.

Do not run them until the kernel, memory, permissions, and validation rules exist.

PHASE 7: FIRST SAFE EXECUTION



Run the first practical skill:

readonly-inventory

Its job:

scan only

map only

report only

no moving

no deleting

no editing

no "cleanup"

PHASE 8: UPDATE SOURCE OF TRUTH



Use the readonly-inventory findings to update:

PATHS.md

CURRENT_STATE.md

SCRATCH_MEMORY.md

QUARANTINE_MEMORY.md

Only promote verified facts into:

PERSISTENT_MEMORY.md

PHASE 9: HERMES / OPENCLAW INTEGRATION



Only now connect Hermes and workers into the rails.

Hermes can operate approved skills.

Workers can execute bounded task cards.

Fable remains architect and judge.

PHASE 10: DASHBOARD / CHAIR / CONTROL SURFACE



Build the UI last.

The dashboard must reflect real backend state.

No fake data.

No pretty control panel before the OS has truth underneath it.

Yes, the build order is the primary file creation order, with execution order. The right

```
Foundation first.  
Kernel second.  
Memory third.  
Router and permissions fourth.  
Validation fifth.  
Worker contracts sixth.  
Skills seventh.  
Read-only inventory eighth.  
Hermes/workers ninth.  
Dashboard last.
```



Yes, the intake pack supports this shape, and the OS starts reading the build order
before the kernel and places some basic kernel memory. Router, Permissions,
Validation, Worker, and System. Hermes, Inventory, Skills, Contracts, then Control
and the dashboard.

gemini-code-1783331857176

... Sources

when u say the master intake is that the file named gemini
code

Thought for a few seconds >

Yes, when I said the master intake pack, I meant the designed

gemini-code-1783331857176.md

What the state is:

MODULE: FABLE-5 MASTER INTAKE PACK

and it contains the master OS structure Fable-5 Master Architect, which defines code
like kernel, OS, OS, system layers, and the phase-build order.

gemini-code-1783331857176

What the state is:

gemini-code-1783331857176.md = clean master intake / OS kernel starter

Pasted text(64).txt = big messy source distillation archive

Use the summary pack file as the base. Use the provided internal evidence source material only.

   ...  Sources